LEXY

ENGINEERING DOCUMENTATION

DOCUMENT 03 / 09

Developer Onboarding Guide

AI Hiring Platform · Intelligence-First

Generated **June 14, 2026** · Confidential — internal use

DOC 03 OF 09 · LEXY ENGINEERING

Lexy — Developer Onboarding Guide

> Engineering documentation set · Doc 3 of 9 Goal: from zero to a running stack, tests green, and a confident first change.

0. Prerequisites & golden rules

- **Node.js 24**, **pnpm** (this repo is pnpm-only — the `preinstall` hook will hard-fail if you run `npm` / `yarn` and will delete stray lockfiles).
- **PostgreSQL** reachable via `DATABASE_URL`.
- TypeScript 5.9.

Golden rule #1 — the build gate is esbuild, not tsc. `tsc` reports ~300 pre-existing type errors (mostly Express request-param typing noise). That is *expected*. Do not try to drive `tsc` to zero — it is not the gate. The api-server ships via `esbuild` (`build.mjs`). Use `tsc` for editor signal, not as a pass/fail.

Golden rule #2 — tenant scoping is not optional. Any query touching tenant-owned data must respect `getAllowedTenantIds`. RLS is a backstop, not an excuse to skip scoping in the handler. See Doc 4 and Doc 6.

1. Install

```
pnpm install          # from repo root; installs the whole workspace
```

2. Configure environment

The API server reads its config from environment variables / Replit secrets. Minimum to boot:

Variable	Purpose
`DATABASE_URL`	PostgreSQL connection string.
`PORT`	HTTP port (Replit assigns one per artifact).
`SESSION_SECRET`	HMAC key for signing session/bearer tokens.
`CORS_ORIGIN`	Allowed cross-origin (empty in dev = same-origin only).

Feature-dependent (add as you touch those areas):

Variable	Enables
`AI_INTEGRATIONS_OPENAI_API_KEY` / `OPENAI_API_KEY`	All LLM features.
`AZURE_SPEECH_KEY` / `AZURE_SPEECH_REGION`	Interview speech-to-text.
`AZURE_TTS_KEY` / `AZURE_TTS_REGION`	Interview text-to-speech.
`AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY` / `AWS_REGION`	S3 (recordings/resumes) + SES (email).
`SES_FROM_EMAIL`	Outbound transactional email sender.
`PDL_API_KEY`, `SERP_API_KEY`	External sourcing providers.

 Never hand-edit secrets or print their values. On Replit, manage them through the environment/secrets tooling. The app degrades gracefully when an optional provider key is missing (the feature is skipped, not crashed).

3. Database setup

- Schema is owned by `@workspace/db` (Drizzle).
- Apply/sync schema with Drizzle Kit push:

```
pnpm --filter @workspace/db run push    # drizzle-kit push (dev sync)
```

- For **production** migrations follow `docs/RUNBOOK_PROD_MIGRATIONS.md` — do not `push` against prod.
- After changing the Drizzle schema, rebuild the db package so downstream tsc stops reporting phantom “no exported member”: `pnpm --filter @workspace/db run build` (i.e. `tsc -b`). *(See memory: “DB package typecheck”.)*

4. Run the stack

Each artifact has a workflow. In Replit they start automatically; from a shell:

```
# Backend (builds via esbuild, then starts dist/index.mjs)
pnpm --filter @workspace/api-server run dev

# Main frontend
pnpm --filter @workspace/lexy run dev

# Public site
pnpm --filter @workspace/lexy-site run dev
```

A healthy backend boot logs `Server listening` plus each scheduler's `Started` line and `[ai-queue] worker started`. The async AI worker can also be run as a separate process: `pnpm --filter @workspace/api-server run worker`.

5. Run tests

Tests use the **native Node test runner via `tsx --test`** (no Jest/Vitest). Named suites in `api-server`:

```
pnpm --filter @workspace/api-server run test:sourcing     # provider adapter layer
pnpm --filter @workspace/api-server run test:fairness     # fairness firewall / PII redaction
pnpm --filter @workspace/api-server run test:learned      # per-tenant learned scoring
pnpm --filter @workspace/api-server run test:similar      # similar-hire embedding signal
pnpm --filter @workspace/api-server run test:global       # cross-tenant global prior
pnpm --filter @workspace/api-server run test:transcribe   # STT pool/admission/transcribe
```

There is no `runTest` subagent in this environment — **test manually** by running the suite above and exercising the relevant flow.

6. Static guardrails (run automatically in `build`)

The api-server `build` step runs two custom linters *before* esbuild — if either fails, the build fails:

- `check:no-console` — no stray `console.*` (use the pino logger).
- `check:no-adverse-writes` — no ungoverned writes to adverse stages (the AI governance guardrail; see Doc 4 / Doc 6).

There's also `check:deletion-cascade-drift` for data-deletion safety.

7. Repo map for your first day

```
artifacts/
  api-server/        ← backend: routes/, lib/agents/, lib/intelligence.ts, schedulers
  lexy/              ← main React app (recruiter + candidate portal)
  lexy-site/         ← marketing site
lib/
  db/                ← Drizzle schema + RLS request proxy   ← READ THIS to understand isolation
  api-zod/           ← shared validation schemas
  api-spec/ api-client-react/  ← Orval codegen + generated hooks
docs/                ← this set lives in docs/engineering/; product guidebooks & runbooks along
.agents/memory/      ← the "why" topic files (extremely high signal — skim the index)
replit.md            ← living project overview + user preferences (respect these)
```

8. Conventions that will bite you if you skip them

- **Frontend queries**: every `useQuery` needs its own `queryFn` (no global default); base var is `BASE` vs `BASE_URL` depending on scope; render a null score as `—`, not `0%`.
- `validate()` **strips unknown body keys**: any new field a handler reads from `req.body` must be added to that route's Zod schema or it is silently dropped.
- **Off-limits files (user preference, do not edit):**
  - `artifacts/lexy/src/components/intelligence/**` — this is the rendering layer for the Intelligence Engine, the product's differentiating spine. It is hand-tuned and owned directly by the project owner; edits here risk subtle regressions to the core scoring UX, so changes must go through them.
  - `artifacts/lexy/src/pages/recruiter/candidates/index.tsx` — the primary recruiter candidate list, the most-trafficked and most carefully tuned screen in the app. It's similarly owner-maintained; coordinate before touching it.

  These are not "broken, stay away" files — they're the highest-stakes, owner-maintained surfaces. Build around them via props/new components rather than editing them in place.
- **Memory first**: before debugging anything non-trivial, skim `.agents/memory/MEMORY.md` — most sharp edges are already documented there with the *why*.

9. Deploy

See `DEPLOY.md` for the full path. In short: backend → a single Node.js Docker container on AWS. There is no committed production orchestrator yet — `DEPLOY.md` documents two interchangeable paths for the same image, **Elastic Beanstalk** (Option A, simpler) or **ECS Fargate** (Option B); confirm which one the environment you're deploying to actually uses before assuming, since their logs and deploy mechanics differ. Frontends → static build to S3 + CloudFront; production DB migrations via `docs/RUNBOOK_PROD_MIGRATIONS.md`. Release checklist: `docs/RELEASE_CHECKLIST.md`.